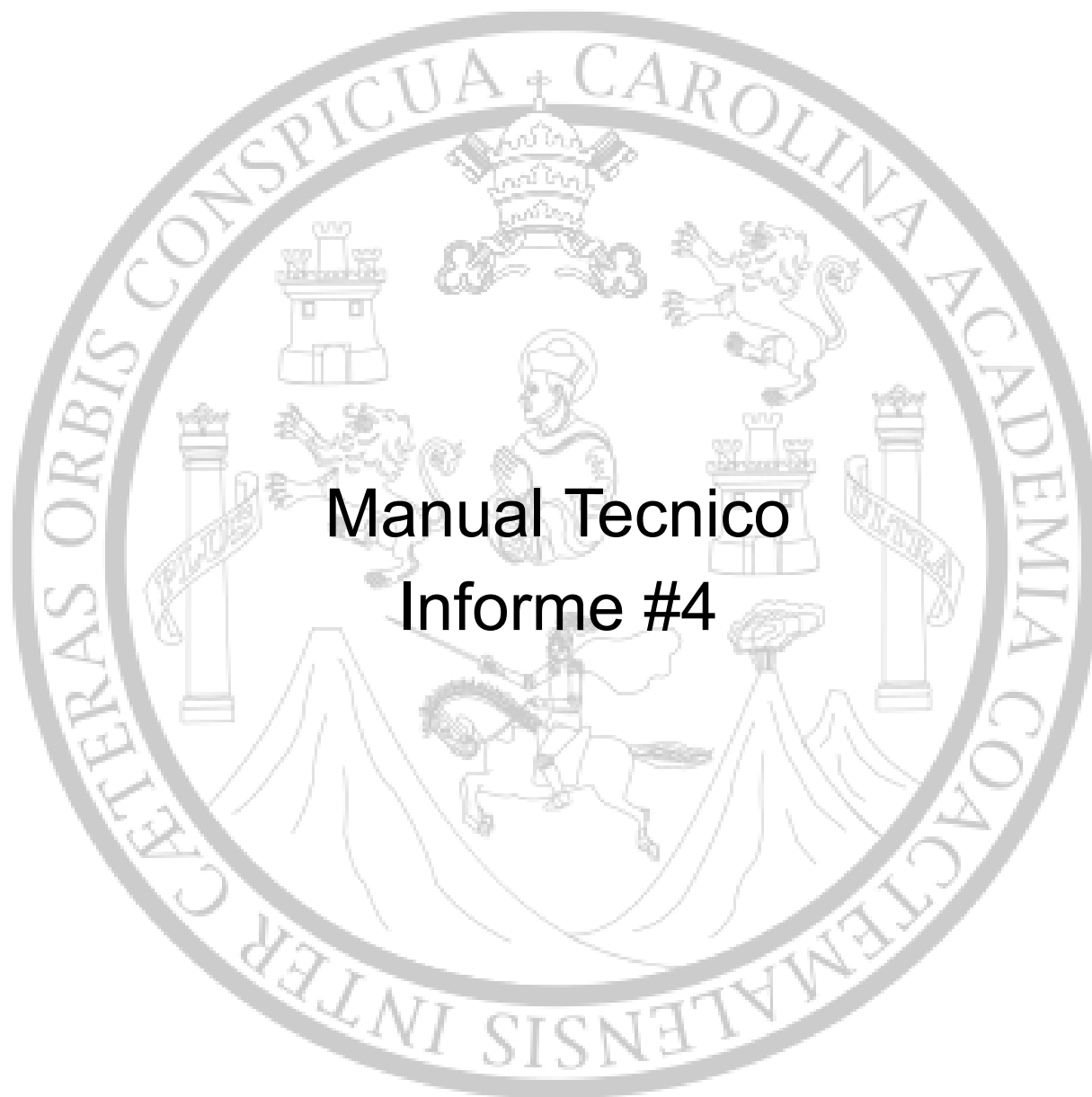


Universidad: San Carlos de Guatemala
Curso: Practicas Iniciales
Sección: C



Manual Tecnico Informe #4

202503403 - Yessenia Crystabel Mazariegos Carranza
202502003 - Luis Emilio Arias Santizo
09 de septiembre del 2026

1. Descripción general

El proyecto utiliza una arquitectura cliente-servidor. El frontend presenta las pantallas al usuario, el backend recibe las solicitudes y aplica las reglas del sistema, y MySQL almacena la información. La comunicación entre frontend y backend se realiza por medio de una API REST.

2. Tecnologías utilizadas

Componente	Tecnología	Uso principal
Frontend	React + Vite	Interfaz web y navegación
Backend	Node.js + Express	API REST y lógica del sistema
Base de datos	MySQL 8	Persistencia de usuarios, publicaciones y cursos
Control de versiones		Ramas, commits y Pull Requests

3. Arquitectura del sistema

El navegador ejecuta el frontend de React. Cuando el usuario realiza una acción, React envía una solicitud HTTP al servidor Express. El backend valida la sesión y los datos recibidos, consulta o modifica MySQL y finalmente devuelve una respuesta JSON al frontend.

Flujo general

Usuario → React/Vite → API REST con Express → MySQL → respuesta JSON → React → Usuario

4. Organización principal del proyecto

El repositorio se divide principalmente en tres carpetas. Esta separación facilita trabajar el cliente, el servidor y la base de datos sin mezclar responsabilidades.

- frontend/: componentes, páginas, rutas y estilos de React.
- backend/: rutas, controladores, middlewares y configuración del servidor.
- database/: estructura de la base de datos, migraciones y datos iniciales.

5. Base de datos

La base de datos utilizada se llama sistema_calificacion_cursos. El modelo es relacional y mantiene las relaciones necesarias para saber qué usuario creó una publicación, qué comentarios pertenecen a cada publicación y qué cursos aprobó cada estudiante.

5.1 Tablas principales

Tabla	Descripción
usuarios	Datos de las cuentas y credenciales del sistema.
cursos	Catálogo de cursos, código y créditos.
catedraticos	Personal académico clasificado como catedrático o auxiliar.

publicaciones	Opiniones creadas por los usuarios.
---------------	-------------------------------------

comentarios	Comentarios asociados a una publicación y a un usuario.
cursos_aprobados	Relación entre usuarios y cursos aprobados.

5.2 Diagrama Entidad-Relación simplificado

usuarios (1) — (N) publicaciones
 usuarios (1) — (N) comentarios
 publicaciones (1) — (N) comentarios
 usuarios (N) — (N) cursos, mediante cursos_aprobados
 cursos (1) — (N) publicaciones
 catedraticos (1) — (N) publicaciones

6. Seguridad y autenticación

Las rutas internas utilizan un token de autenticación. El frontend guarda el token de la sesión y lo envía en el encabezado Authorization. El middleware del backend verifica el token antes de permitir el acceso. Las operaciones de perfil y cursos aprobados utilizan el identificador del usuario autenticado, evitando que una persona modifique directamente la información de otra.

7. Endpoints principales de la API

Método	Ruta	Función
POST	/api/auth/registro	Registrar un usuario.
POST	/api/auth/login	Iniciar sesión.
POST	/api/auth/recuperar-password	Restablecer contraseña.
GET	/api/auth/verificar	Verificar una sesión.
GET	/api/catalogos/cursos	Listar cursos disponibles.
GET	/api/catalogos/personal	Listar catedráticos y auxiliares.
GET	/api/publicaciones	Listar y filtrar publicaciones.
POST	/api/publicaciones	Crear una publicación.
GET	/api/publicaciones/:id/comentarios	Consultar comentarios.
POST	/api/publicaciones/:id/comentarios	Agregar un comentario.
GET	/api/cursos-aprobados	Consultar cursos propios.
POST	/api/cursos-aprobados	Agregar un curso aprobado.
DELETE	/api/cursos-aprobados/:cursold	Eliminar un curso aprobado.
GET	/api/perfil	Consultar el perfil propio.
PUT	/api/perfil	Editar nombres, apellidos y correo.
GET	/api/perfil/:registro	Consultar un perfil público.

8. Instalación y ejecución local

8.1 Requisitos

- Node.js y npm instalados.
- MySQL Server 8 instalado y en ejecución.
- Git instalado para clonar y actualizar el repositorio.

8.2 Base de datos

Crear la base de datos y ejecutar los archivos SQL incluidos en la carpeta database. Las migraciones deben aplicarse en orden y luego pueden cargarse los datos iniciales del catálogo.

8.3 Backend

Entrar a la carpeta backend, instalar dependencias con npm install y configurar el archivo .env con los datos de conexión a MySQL y la clave utilizada para los tokens. Después iniciar el servidor con el script definido en package.json. Durante las pruebas locales el backend se utilizó en el puerto 3000.

8.4 Frontend

Entrar a la carpeta frontend, ejecutar npm install y luego npm run dev. Vite inicia la aplicación para desarrollo. En las pruebas locales se utilizó el puerto 5173. Para comprobar la compilación final se puede ejecutar npm run build.

9. Control de versiones

El trabajo se organizó con las ramas main, develop y ramas feature. Los cambios de cada módulo se guardaron en commits separados y se integraron mediante Pull Requests. Al finalizar, develop se unió a main mediante el Pull Request final.

10. Pruebas realizadas

- Registro, inicio de sesión y validación de sesión.
- Creación y persistencia de publicaciones.
- Filtros por texto, curso y personal académico.
- Creación y consulta de comentarios.
- Agregar y eliminar cursos aprobados y cálculo de créditos.
- Consulta y edición del perfil propio.
- Búsqueda de estudiantes por registro académico.
- Consulta de perfiles públicos en modo solo lectura.
- Compilación del frontend con Vite para producción.

11. Observaciones finales

El sistema cumple el flujo principal solicitado para el MVP. El frontend, backend y base de datos trabajan de forma integrada. Para una demostración se recomienda iniciar primero MySQL, luego el backend y finalmente el frontend, y utilizar datos de prueba que permitan mostrar publicaciones, comentarios, perfiles y cursos aprobados.